# Predicción de daño estructural post-terremoto

**Actividad 5 — Avance del proyecto: modelo base entrenado**

Richter's Predictor: Modeling Earthquake Damage (DrivenData) · Fundamentos de Redes Neuronales — Ciencia de Datos — Universidad de La Sabana
Equipo: Ana Ferrer · Sofía Nuñez · Valentina Ramirez

## 1. Recordatorio del problema

El 25 de abril de 2015, un terremoto de magnitud 7.8 con epicentro en Gorkha, Nepal, dejó cerca de 9,000 personas fallecidas y dañó o destruyó más de 600,000 estructuras. El proyecto busca responder: **¿es posible predecir el nivel de daño estructural (bajo, medio o casi total) que sufrirá un edificio ante un terremoto, a partir de sus características constructivas, los materiales utilizados y la zona geográfica en la que se ubica?**

- **Variable objetivo:** *damage_grade*, ordinal, 3 categorías (1 = daño bajo, 2 = daño medio, 3 = daño casi total).
- **A quién beneficia:** autoridades de gestión de riesgo, entidades de reconstrucción, aseguradoras y comunidades, porque permite priorizar inspecciones y focalizar el refuerzo sísmico con recursos limitados.
- **Por qué una red neuronal:** hay relaciones no lineales e interacciones entre variables (p. ej. el tipo de cimentación es más riesgoso en edificios de mayor altura) que un modelo lineal no captura sin construir manualmente esos términos de interacción.

Esta entrega consolida el primer paso técnico: construir, entrenar y evaluar un modelo base funcional que valide la viabilidad técnica del enfoque propuesto en la Unidad 1. No se busca el mejor desempeño posible, sino evidencia clara de implementación y capacidad de diagnóstico crítico.

## 2. Descripción del dataset

**Fuente.** Encuestas post-desastre de Kathmandu Living Labs y la Central Bureau of Statistics de Nepal, empaquetadas por DrivenData para la competencia Richter's Predictor (2018).

**Tamaño.** 260,601 edificaciones de entrenamiento × 39 variables predictoras, más 86,868 edificaciones de prueba sin etiqueta reservadas por la competencia. Sin valores nulos ni duplicados (verificado en la Unidad 1).

**Calidad y preprocesamiento.** El preprocesamiento se congeló en disco antes de este notebook (*terremoto_train_procesado.parquet + preprocesamiento_metadata.json*) para no repetir la limpieza en cada ejecución y garantizar que el encoding se calcule siempre sobre las mismas filas. Decisiones clave:

- **age = 995** (1,390 registros): no es una edad real sino un valor especial no documentado por DrivenData. Se creó una bandera binaria (*age_valor_especial_995*) y se imputó con la mediana real (15 años), en vez de descartar los registros.
- **geo_level_1_id** (31 categorías) y las 8 variables categóricas restantes con *one-hot encoding* (69 columnas).
- **geo_level_2_id** (1,414 categorías) y **geo_level_3_id** (11,595 categorías) con *target encoding* suavizado, que evita sobreajustar en categorías con pocas observaciones. Se ajustó solo con el split de entrenamiento para no filtrar información del target hacia validación. En las soluciones públicas de esta competencia las variables *geo_level_** aparecen consistentemente entre las más predictivas, así que su codificación es una decisión central del pipeline; esto se verifica empíricamente en la Sección 4.2.
- Variables numéricas estandarizadas con *StandardScaler* (ajustado solo con train) y 23 variables binarias (*has_superstructure_**, *has_secondary_use_**) sin transformar.
- **Split train/val 80/20 estratificado**, fijado desde el preprocesamiento porque el target encoding depende de qué filas son "train".

Resultado: **99 columnas de features**, sin nulos, 208,480 registros de entrenamiento y 52,121 de validación, con la distribución de clases preservada exactamente entre ambos splits (9.64% / 56.89% / 33.47%).

### 2.1 Análisis exploratorio

Antes de entrenar conviene ver las relaciones que motivan el uso de una red neuronal, en vez de solo describirlas. Para estas visualizaciones se usaron las variables originales sin codificar, porque las versiones one-hot y target-encoded del dataset procesado no son legibles en un gráfico exploratorio.

Figura 1 — Exploración de la variable objetivo y dos predictoras clave

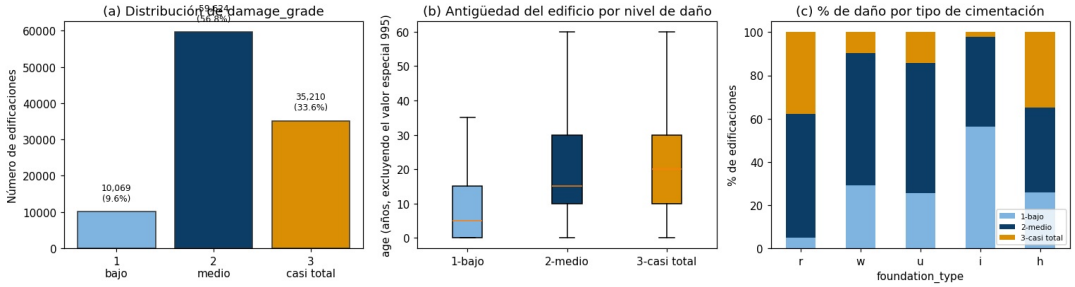



*Figura 1 — Distribución del target, antigüedad por nivel de daño y tipo de cimentación por nivel de daño.*

**Lectura.** El panel (a) confirma el desbalance moderado ya documentado. El panel (b) muestra una asociación clara entre antigüedad y daño: la mediana de edad crece visiblemente del nivel 1 al nivel 3, consistente con edificios más viejos, construidos bajo normas sismorresistentes menos exigentes o ya deterioradas. El panel (c) muestra que el tipo de cimentación por sí solo separa fuertemente el riesgo: algunas categorías concentran más del 50% de sus edificios en daño bajo, mientras otras concentran una proporción similar en daño casi total. Esa es exactamente la relación no lineal y dependiente de la categoría que motivó, en la Unidad 1, descartar un modelo puramente lineal.

## 3. Arquitectura y configuración del modelo base

Se eligió un perceptrón multicapa (MLP) deliberadamente simple, dejando el ajuste fino de hiperparámetros para la siguiente iteración. El modelo se entrenó época a época mediante *partial_fit* con *warm_start*: cuesta más tiempo de cómputo que un único *fit()*, pero es la única forma de registrar pérdida, exactitud y F1 de validación en cada época y poder graficar curvas de aprendizaje reales en lugar de solo la pérdida de entrenamiento.

| Decisión | Valor elegido | Justificación |
|---|---|---|
| Capas ocultas | (64, 32) | Punto de partida proporcional a 99 variables de entrada y 3 clases de salida: ni tan pequeño que no pueda capturar interacciones, ni tan grande que sea difícil de entrenar y costoso. |
| Activación | ReLU | Estándar para capas ocultas; evita la saturación del gradiente que sí presentan sigmoide y tanh. |
| Optimizador | Adam (lr = 0.001) | Convergencia rápida sin necesidad de ajustar finamente la tasa de aprendizaje. |
| Regularización | alpha = 0.0001 | Valor por defecto, deliberadamente bajo por tratarse de un modelo base; el ajuste de regularización queda como oportunidad de mejora (Sección 7). |
| Función de pérdida | Entropía cruzada categórica (softmax en la salida) | Estándar para clasificación multiclase. Nótese que *damage_grade* es ordinal y no nominal, una limitación que se retoma en la Sección 7. |
| Épocas | 60 | Suficientes para observar el comportamiento completo del entrenamiento (convergencia y posible sobreajuste) sin un costo excesivo. |
| Semilla | random_state = 42 | Fijada en todo el notebook para que los resultados sean reproducibles. |

## 4. Métricas de entrenamiento y validación

La métrica oficial de la competencia es el **F1 micro-promediado**. Por el desbalance de clases documentado en la Sección 2, se reporta también el **F1 macro**, que promedia el F1 de cada clase por igual sin ponderar por su tamaño; la brecha entre ambos es justamente lo que revela el problema de la clase minoritaria.

| 0.7345 | 0.6795 | 0.6011 | 0.7624 |
|---|---|---|---|
| F1 MICRO (VAL) | F1 MACRO (VAL) | PÉRDIDA (VAL) | EXACTITUD (TRAIN) |

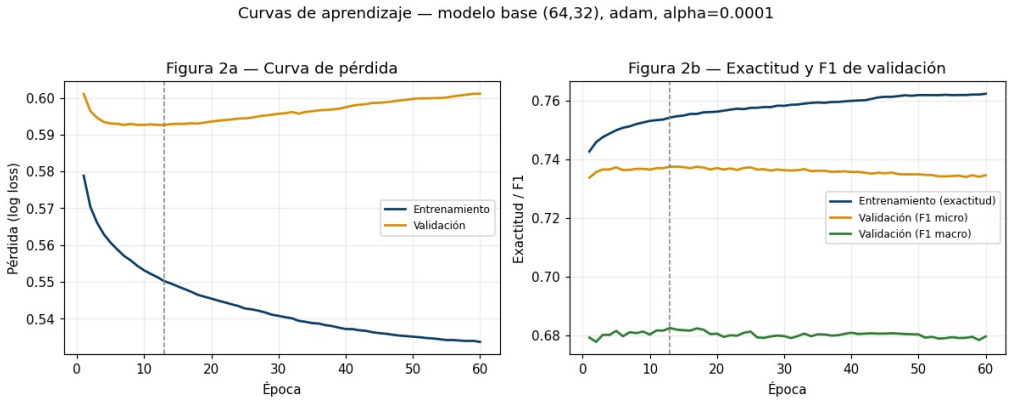



*Figura 2 — Curvas de aprendizaje del modelo base. Izquierda: pérdida de entrenamiento vs. validación. Derecha: exactitud de entrenamiento y F1 micro/macro de validación. La línea punteada marca la época 13, mínimo de la pérdida de validación.*

El mínimo de la pérdida de validación es **0.5926 en la época 13**; al llegar a la época 60 la pérdida de entrenamiento baja a 0.5337 mientras la de validación sube a 0.6011. Es decir, el modelo sigue aprendiendo del conjunto de entrenamiento después de la época 13, pero ese aprendizaje adicional ya no se traduce en mejor desempeño sobre datos no vistos.

### 4.1 Desempeño por clase

```
                    precision    recall  f1-score   support

      1 - Daño bajo      0.658     0.501     0.569      5025
     2 - Daño medio      0.741     0.829     0.782     29652
3 - Daño casi total      0.740     0.641     0.687     17444

          accuracy                          0.735     52121
         macro avg      0.713     0.657     0.680     52121
      weighted avg      0.733     0.735     0.730     52121
```

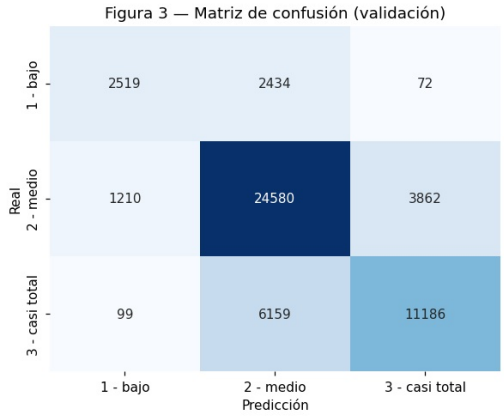



*Figura 3 — Matriz de confusión sobre el conjunto de validación (52,121 edificaciones).*

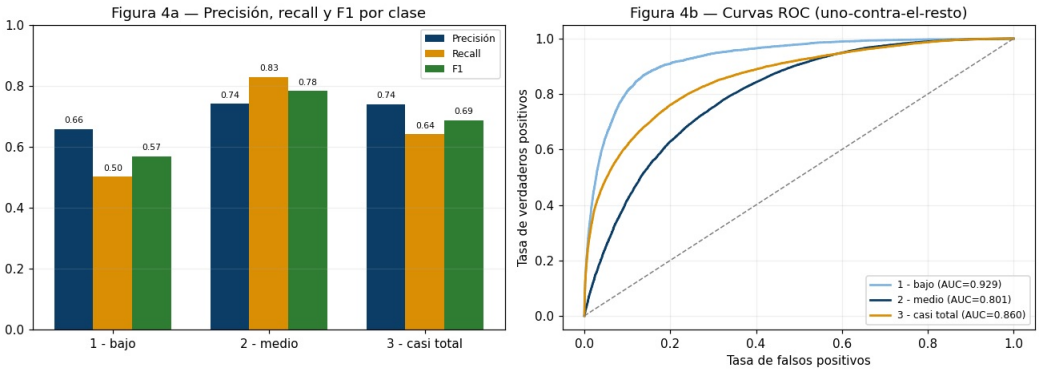



*Figura 4 — Precisión, recall y F1 por clase (izq.) y curvas ROC uno-contra-el-resto con su AUC (der.).*

**Interpretación.** El modelo acierta mejor en la clase mayoritaria (daño medio, recall 0.829) y peor en la minoritaria (daño bajo, recall 0.501): pierde prácticamente la mitad de los edificios que realmente tuvieron daño bajo, confundiéndolos sobre todo con daño medio. La precisión, en cambio, es relativamente pareja entre las tres clases (0.66–0.74), lo que indica que cuando el modelo *sí* predice clase 1 suele acertar; el problema es que la predice poco. Las curvas ROC matizan el diagnóstico: incluso la clase 1 alcanza un AUC alto, de modo que el modelo sí ordena bien los casos por riesgo relativo. El problema está más en dónde queda el umbral de decisión de *argmax* bajo desbalance que en la capacidad de separación del modelo, y eso cambia qué tipo de solución tiene sentido probar después.

### 4.2 Importancia de variables

Para saber qué variables está usando realmente el modelo se calculó *permutation importance*: se permuta aleatoriamente una variable a la vez sobre una muestra de 8,000 registros de validación (8 repeticiones) y se observa cuánto cae el F1 micro. Se usó una muestra y no los 52,121 registros para mantener el costo computacional razonable conservando la barra de error.

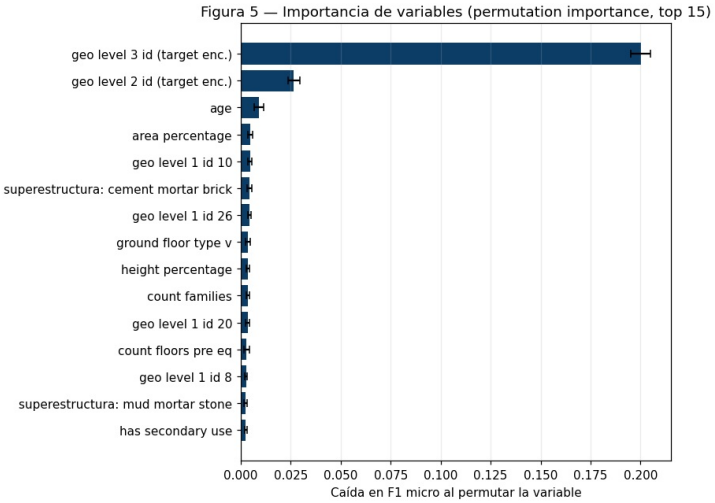



*Figura 5 — Top 15 de variables por caída en F1 micro al permutarlas.*

**Confirmación empírica.** Las dos variables más importantes son las codificaciones de *geo_level_3_id* (caída de 0.200 en F1 micro) y *geo_level_2_id* (0.027), exactamente lo que reportan las soluciones públicas consultadas en la Unidad 1. Les siguen la antigüedad, el área y variables de material de la superestructura. Esto valida dos decisiones del pipeline: el target encoding sobre las variables geográficas efectivamente capturó una señal fuerte, y la ubicación geográfica es —tal como plantea el problema— un factor central del riesgo estructural en Nepal. El salto de un orden de magnitud entre la primera y la segunda variable también advierte que el modelo depende mucho de una sola representación, lo que es un riesgo a considerar en la siguiente iteración.

## 5. Diagnóstico técnico del comportamiento del modelo

**Convergencia.** El modelo converge rápido: la pérdida de validación toca su mínimo alrededor de la época 13 (Figura 2a). Adam, con su tasa de aprendizaje adaptativa, encuentra una buena región de la superficie de error en pocas iteraciones, consistente con lo observado en el proyecto anterior sobre un dataset mucho más pequeño.

**Estabilidad.** Las curvas son suaves, sin oscilaciones ni picos: no hay señales de una tasa de aprendizaje demasiado alta ni de inestabilidad numérica. Es esperable dado el tamaño del conjunto de entrenamiento (208,480 muestras): con lotes de datos tan grandes, las estimaciones del gradiente en cada paso son mucho menos ruidosas que en un dataset pequeño, y por eso las curvas se ven tan limpias.

**Generalización.** Después de la época 13 aparece un sobreajuste leve pero presente: la pérdida de entrenamiento sigue bajando (de 0.55 a 0.53) mientras la de validación sube ligeramente (de 0.593 a 0.601). La brecha final entre exactitud de entrenamiento (0.762) y F1 micro de validación (0.735) es de menos de 3 puntos, así que el sobreajuste existe pero no es el factor que limita el desempeño.

**Comportamiento por clase.** La exactitud global del 73.5% esconde una brecha real: la clase minoritaria (daño bajo, 9.6% de los datos) tiene un recall de apenas 50.1%, y 2,434 de sus 5,025 casos se confunden con daño medio (Figura 3). Esa asimetría es lo que produce la diferencia de casi 6 puntos entre F1 micro (0.735) y F1 macro (0.680). Como las curvas ROC muestran buena separabilidad incluso para la clase 1, el diagnóstico es que **el cuello de botella no es la capacidad del modelo sino el desbalance de clases y el umbral de decisión que induce**. Esa hipótesis es precisamente lo que se pone a prueba en la Sección 6.

## 6. Experimentación adicional: ¿se puede mejorar con ajustes simples?

En vez de enunciar oportunidades de mejora, se probaron dos de forma rápida y controlada, para ver si un ajuste simple ya cambia el diagnóstico. Los tres modelos se entrenaron a **30 épocas** —incluida una base recalculada— para que la comparación fuera justa, y el sobremuestreo se aplicó únicamente al conjunto de entrenamiento, nunca a validación, para no inflar artificialmente el desempeño medido.

| Configuración (30 épocas) | F1 micro | F1 macro | Recall clase 1 | Precisión clase 1 |
|---|---|---|---|---|
| Base (64, 32) | 0.7344 | 0.6905 | 0.5652 | 0.6324 |
| Arquitectura grande (128, 64) | 0.7332 | 0.6849 | 0.5538 | 0.6202 |
| Sobremuestreo clase 1 (64, 32) | 0.6890 | 0.6616 | 0.8020 | 0.4441 |

*Tabla 1 — Comparación de experimentos adicionales sobre el mismo split de validación.*

Figura 6 — Experimentación adicional: F1 micro, F1 macro y recall de la clase 1

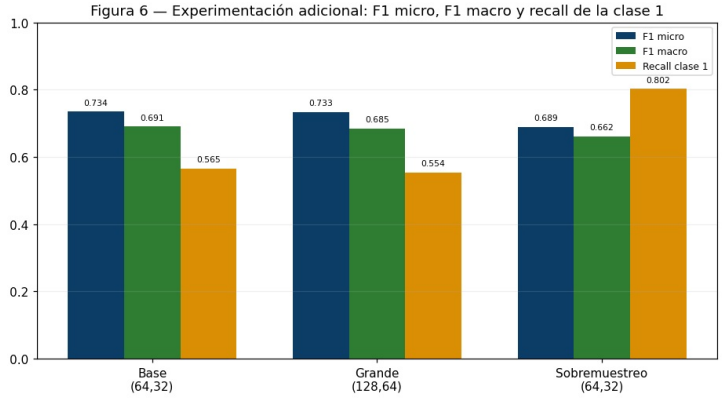



*Figura 6 — F1 micro, F1 macro y recall de la clase 1 para las tres configuraciones.*

**Arquitectura más grande (128, 64).** No ayuda: F1 micro y F1 macro quedan prácticamente iguales a la base, incluso una fracción más bajos, y el recall de la clase 1 tampoco mejora. El diagnóstico de la Sección 5 ya apuntaba a que el problema no es falta de capacidad del modelo sino el desbalance de clases; este experimento lo confirma, porque agregar neuronas no ataca esa causa.

**Sobremuestreo de la clase 1.** El recall de la clase 1 sube de 0.565 a 0.802, así que la técnica sí logra lo que promete. Pero el F1 micro cae de 0.734 a 0.689 y el F1 macro baja de 0.691 a 0.662. La precisión de la clase 1 se desploma de 0.632 a 0.444 y, más importante, el recall de la clase 2 (la mayoritaria) cae de 0.795 a 0.644. Al ver la clase 1 tres veces más de lo normal, el modelo se vuelve más propenso a predecirla también cuando no corresponde, y eso le cuesta acierto en la clase mayoritaria. La lección es que una técnica que mejora la métrica mirada de forma aislada puede empeorar el balance general si no se mide su efecto sobre las demás clases.

## 7. Oportunidades de mejora para la siguiente iteración

Con la evidencia de la Sección 6, estas son las oportunidades priorizadas:

1. **Tratar *damage_grade* como lo que es: una variable ordinal, no nominal.** La pérdida actual (softmax + entropía cruzada) penaliza igual confundir la clase 1 con la 3 que confundirla con la 2, cuando en la práctica el segundo error es mucho menor. Un enfoque ordinal debería mejorar el F1 macro sin el costo colateral que sí tuvo el sobremuestreo simple.
2. **Buscar un punto intermedio de sobremuestreo.** En vez del balanceo completo, sobremuestrear la clase 1 solo parcialmente o combinarlo con ponderación de la función de pérdida, para recuperar parte del recall de la clase 1 sin sacrificar tanto la clase 2.
3. **Diseñar experimentos controlados de regularización.** El modelo base usa alpha = 0.0001 sin ninguna búsqueda, y el sobreajuste leve de la Sección 5 sugiere margen ahí.
4. **Explorar *embeddings* aprendidos para las variables geográficas** en lugar de target encoding: la Sección 4.2 confirmó que son las más importantes del modelo, así que una mejor representación podría tener más impacto que cualquier otro ajuste.
5. **Comparar contra modelos de árbol** (Random Forest, XGBoost, LightGBM). La literatura pública de esta competencia reporta que suelen superar a redes neuronales simples (F1 micro de 0.72–0.76), y sirven como referencia de qué tan lejos está el MLP del techo alcanzable con este dataset.
6. **Usar un criterio de detención más robusto** que un número fijo de épocas, por ejemplo *early stopping* sobre la pérdida de validación, que el propio diagnóstico ya sitúa en la época 13.

## 8. Trabajo colaborativo

El trabajo se repartió en tres bloques técnicos de carga equivalente, uno por integrante, siguiendo el orden natural del pipeline (datos → modelo → evaluación). Cada integrante implementó su bloque y justificó las decisiones tomadas dentro de él, y además revisó el bloque de otra compañera antes de integrarlo, de modo que ninguna sección quedó validada únicamente por quien la escribió. El diagnóstico (Sección 5) y las conclusiones se redactaron de forma conjunta, porque dependen de los tres bloques a la vez.

| Integrante | Responsabilidad en esta entrega | Decisiones que tomó / justificó |
|---|---|---|
| Ana Ferrer | Datos y análisis exploratorio (Secciones 2 y 2.1): consolidación del dataset procesado, verificación de calidad y construcción de la Figura 1. | Tratar *age = 995* como valor especial (bandera binaria + imputación por mediana) en vez de eliminar los 1,390 registros. Usar target encoding suavizado para *geo_level_2/3_id* en lugar de one-hot, que habría hecho explotar la dimensionalidad. Ajustar encoding y escalado solo con train para no filtrar información del target. Congelar el split 80/20 estratificado en disco para que el encoding no cambie entre ejecuciones. |
| Sofía Nuñez | Modelo base y curvas de aprendizaje (Secciones 3 y 4): función de entrenamiento, ejecución del MLP y Figura 2. | Entrenar época a época con *partial_fit* en vez de un único *fit()*: cuesta más cómputo, pero es lo que permite registrar métricas de validación por época y graficar curvas reales. Arquitectura (64, 32) proporcional a las 99 variables de entrada, dejando explícita la búsqueda de hiperparámetros para la siguiente iteración. Reportar F1 micro *y* F1 macro, porque con el desbalance la exactitud sola habría ocultado el problema de la clase 1. Fijar semilla en todo el notebook. |
| Valentina Ramirez | Evaluación por clase, diagnóstico y experimentación (Secciones 4.1, 4.2, 5, 6 y 7): matriz de confusión, curvas ROC, importancia de variables y experimentos. | Complementar la matriz de confusión con curvas ROC uno-contra-el-resto, que fue lo que permitió distinguir un problema de umbral de un problema de separabilidad. Calcular *permutation importance* sobre una muestra de 8,000 registros para mantener el costo razonable conservando la barra de error. Entrenar los tres modelos comparados a 30 épocas para que la comparación fuera justa. Aplicar el sobremuestreo solo a train, nunca a validación. |

**Reparto de la carga.** Los tres bloques tienen una carga comparable, cerca de un tercio del tiempo total de la actividad cada uno. Ninguna integrante concentró el código y ninguna quedó solo con tareas de redacción: cada una escribió código, ejecutó su parte y redactó la interpretación de sus propios resultados. Las revisiones cruzadas se distribuyeron en ciclo (Ana revisó el bloque de Valentina, Valentina el de Sofía y Sofía el de Ana) y las decisiones que afectaban a más de un bloque se acordaron en sesiones conjuntas antes de implementarlas, no después.

> **Decisiones del equipo documentadas.**
>
> 1. **No re-ejecutar el preprocesamiento dentro del notebook.** Se congeló el resultado de la Unidad 1 en disco; además de ahorrar tiempo, garantiza que el target encoding se calcule siempre sobre exactamente las mismas filas de entrenamiento.
> 2. **No perseguir el mejor F1 posible en este hito.** La actividad pide evidencia de implementación y diagnóstico crítico, así que se priorizó un modelo simple y bien entendido sobre uno ajustado cuyo comportamiento no pudiéramos explicar.
> 3. **Probar las oportunidades de mejora en vez de solo enunciarlas** (Sección 6): dos experimentos controlados con resultado medible antes que una lista larga de propuestas sin evidencia.
> 4. **Reportar el resultado negativo del sobremuestreo tal cual salió**, en lugar de descartarlo, porque es justamente lo que orienta la siguiente iteración.
> 5. **Uso de IA** limitado a validación de sintaxis de código y revisión conceptual de la redacción. La implementación, la interpretación de las métricas y la escritura del informe son propias del equipo.

## 9. Conclusiones

Se implementó y entrenó un modelo base (MLP 64-32) sobre el dataset procesado de la Unidad 1, alcanzando un **F1 micro de 0.735** sobre 52,121 edificaciones de validación, dentro del rango reportado por soluciones públicas de esta misma competencia para modelos sin ajuste fino. El modelo converge de forma rápida y estable, con un sobreajuste leve a partir de la época 13. El hallazgo más relevante del diagnóstico es que el desbalance de clases afecta el desempeño de forma desigual: la clase minoritaria (daño bajo) tiene un recall considerablemente menor que las otras dos, lo que queda cuantificado en la brecha entre F1 micro y F1 macro y matizado por las curvas ROC, que muestran que la separabilidad sí existe y el problema está en el umbral de decisión.

Más allá del modelo base se probaron dos ajustes simples y ninguno resultó una mejora sin costo: la arquitectura más grande no cambió nada, confirmando que el cuello de botella no es la capacidad del modelo, y el sobremuestreo mejoró mucho el recall de la clase 1 pero a costa del desempeño general. La lección es que las intervenciones deben evaluarse por su efecto completo y no solo en la métrica que motivó probarlas. La importancia de variables confirmó empíricamente que las variables geográficas son las más determinantes del riesgo estructural en este dataset, validando el target encoding aplicado en el preprocesamiento.

El modelo base cumple su propósito: demuestra la viabilidad técnica del enfoque propuesto en la Unidad 1 y deja un diagnóstico claro, respaldado por evidencia experimental, de por dónde debe empezar la siguiente iteración: tratamiento ordinal del target y un manejo más cuidadoso —no solo más agresivo— del desbalance de clases.

## Referencias


- Kathmandu Living Labs, & Central Bureau of Statistics, National Planning Commission Secretariat of Nepal. (2018). *Richter's Predictor: Modeling earthquake damage* [Dataset]. DrivenData. https://www.drivendata.org/competitions/57/nepal-earthquake/
- Aggarwal, C. C. (2023). *Neural Networks and Deep Learning: A Textbook* (2nd ed.). Springer.
- Bishop, C. M., & Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.
- Géron, A. (2017). *Hands-On Machine Learning with Scikit-Learn and TensorFlow*. O'Reilly Media.